

Performance Testing

Date	30 June 2026
Team ID	xxxxxx
Project Name	EduGenie Google Gemini Powered Learning Assistant
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Testing
Type of Testing	Functional Testing, Performance Testing
Target Module / API	/qa, /summary, /quiz, /learning, /image, /pdf
Test Environment	Local Machine (Windows 11, FastAPI, Uvicorn)
Test Date	2 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Question Answering API	1	30	AI generates accurate answers
2	Image Analysis API	1	40	Image processed successfully
3	PDF Chat API	1	60	PDF analyzed and questions answered
4	Quiz Generation API	1	30	Quiz generated successfully

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.4	Pass	Average response within limit
2	Response Time (Max)	< 5 seconds	4.2	Pass	Large prompts require more time
3	Throughput (Req/sec)	3 Req/sec	3 Req/sec	Pass	Stable local performance
4	Error Rate	< 1%	0%	Pass	No errors during testing
5	CPU Utilization	< 80%	55%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	68%	Pass	Memory usage within limit

Step 4: Observations & Analysis

Key Findings

- AI features responded correctly during testing.
- Question answering, summarization, quiz generation, and learning recommendations worked successfully.
- PDF and image processing modules functioned correctly.
- Overall application remained stable throughout testing.

Bottlenecks Identified

- Initial loading of the LaMini-Flan-T5 model increases startup time.
- PDF processing takes longer for larger documents.
- Gemini API response depends on internet speed and API latency.

Optimization Steps Taken

- Used FastAPI for efficient request handling.
- Reused loaded AI models to reduce initialization time.
- Optimized PDF text extraction for repeated queries.
- Improved frontend responsiveness and chat interaction.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

